

StackSense — Technical Architecture & Build Spec

Oct 3, 2026 · @Arman

1. Overview, goals and non-goals

StackSense is a web app that runs an adaptive health intake, turns the answers into a personalised supplement stack, maps the stack's combined effect across eight body areas, schedules exact doses on a calendar, and links each item to a top-rated product through affiliate links. The companion prototype spec walks one user (Maya) through every screen; this document is what engineers and AI coding agents build from.

The core design decision: **recommendations come from a deterministic, auditable rules-and-evidence engine; the LLM only phrases questions, explains results and suggests follow-ups inside guardrails.** No dose, exclusion or interaction ever originates from model output.

The business runs on a freemium model: a one-time Full Report, a Plus subscription and affiliate commissions (section 13). With consent, each user's answers, labs, plans and results are stored as a health profile that makes their next recommendation more personal (section 14). Staff manage user data, recommendation rules, affiliate links and pricing from an admin panel (section 15).

Goals (v1)

#	Goal	Measure
G1	Intake that adapts after every answer	Median 15–25 cards; ≥ 3 visible branch events per session
G2	Safe, explainable stack	100% of items and exclusions trace to rules + citations; 0 known contraindicated items shipped
G3	Health Impact Map	Scores per body area computed and reproducible from stored inputs
G4	Dosage calendar	Correct slots for $\geq 99\%$ of plans in property tests; .ics export
G5	Monetisation	Affiliate link on every product row; Full Report and Plus purchasable; revenue per completed intake tracked

#	Goal	Measure
G6	Performance	Next-question latency p95 < 300 ms (rules path), < 1.5 s (LLM phrasing path)
G7	Consented personalisation	100% of stored health data tied to a consent record; returning users answer $\geq 40\%$ fewer cards
G8	Admin panel	Rules, products, links, prices and paywall gates changeable without a code release; every change audited

Non-goals (v1)

- Diagnosing conditions or replacing a clinician; prescription drugs; dosing for children or pregnancy (restricted flows only).
- Selling products directly (no cart or inventory).
- Wearable / lab-API integrations (manual lab entry only; integrations in v2).

Glossary

Term	Meaning
Signal	A scored hypothesis about the user, e.g. <code>low_iron_risk = 0.78</code> , created by answers
Question node	A card in the question graph with preconditions, an answer schema and signal effects
Ingredient	A supplement compound (magnesium bisglycinate), independent of brand
Product (SKU)	A buyable item that contains one or more ingredients at stated amounts
Body area	One of 8 outcome axes: Sleep, Energy, Joints & recovery, Mood & stress, Performance, Skin & hair, Immunity, Heart & metabolic
Plan	A versioned stack + schedule generated for one user at one time
Entitlement	A feature key a user has unlocked by purchase or subscription, e.g. <code>exact_doses</code>

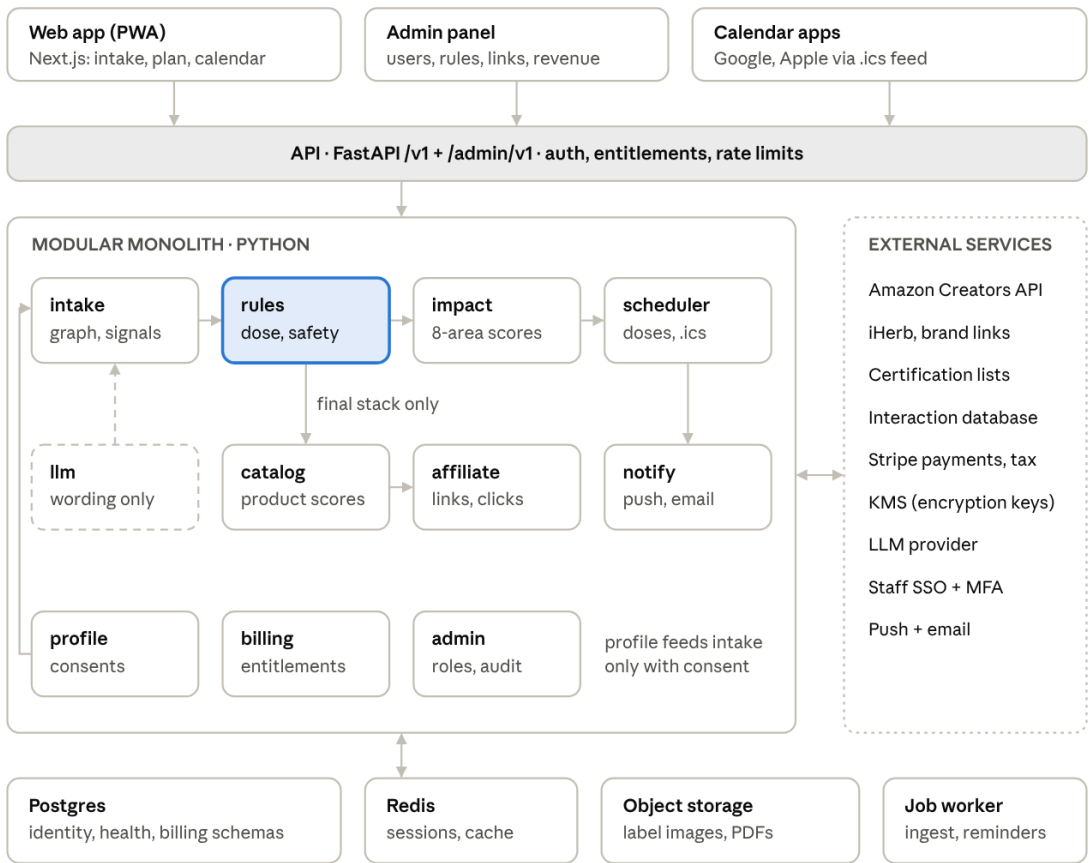
Term	Meaning
Consent purpose	One named use of personal data (e.g. personalisation) that the user grants or withdraws separately
Health profile	The consented, versioned history of a user's answers, labs, plans, adherence and outcomes

2. System architecture

v1 is one web client, one API and a modular monolith with a job worker. Modules talk through in-process interfaces with the same contracts they would have as services, so any of them can be split out later.

One API in front of a modular monolith; the rules engine decides, the LLM only words

v1 runs as one deployable plus a job worker; billing, consent and admin are modules, not separate systems



StackSense v1 architecture · 11 modules, 4 data stores

A request flows top to bottom: the PWA posts an answer, intake updates signals and returns the next card; at the end rules builds the stack, impact and scheduler read it, and only then do catalog and

affiliate attach products and links. The `11m` module can reword text going to the user but has no write path into rules or plans.

Three modules support the business around that flow: `profile` holds consents and the health history and feeds returning users' intake only with consent; `billing` turns Stripe events into entitlements that the API checks before returning gated fields; `admin` serves the admin panel with role scopes and an audit log. None of the three can change what `rules` recommends.

3. Adaptive intake engine

The intake is a **question graph plus a signal model**. After each answer the engine updates signals, then picks the next question that most reduces uncertainty about the signals that matter for this user's goals. Clinicians and content editors own the graph as versioned data; engineers own the engine.

3.1 Question node schema

```
{
  "id": "B2_iron_cluster",
  "version": 3,
  "phase": "follow_signals",
  "prompt": "Do you notice any of these?",
  "why": "These can point to low iron or B12, which change what we recommend.",
  "answer": { "type": "multi_select", "options": [
    { "id": "breathless_stairs", "label": "Short of breath on stairs" },
    { "id": "cold_extremities", "label": "Cold hands or feet" },
    { "id": "hair_shedding", "label": "More hair shedding" },
    { "id": "tingling", "label": "Tingling hands or feet" },
    { "id": "winter_low_mood", "label": "Low mood in winter" } ] },
  "allow_unsure": true },
  "preconditions": "signals.fatigue >= 0.4 OR goals contains 'energy'",
  "targets": ["low_iron_risk", "b12_risk", "vitamin_d_risk"],
  "effects": [
    { "if": "breathless_stairs", "signal": "low_iron_risk", "lr": 2.0 },
    { "if": "hair_shedding", "signal": "low_iron_risk", "lr": 1.6 },
    { "if": "tingling", "signal": "b12_risk", "lr": 2.2 },
    { "if": "winter_low_mood", "signal": "vitamin_d_risk", "lr": 1.4 } ],
  "red_flags": [],
  "evidence_refs": ["ev_iron_symptoms_01"],
  "locale": { "fr-CA": { "prompt": "..." } }
}
```

Key fields: `preconditions` (a small safe expression language, never eval), `targets` (signals this question informs), `effects` as likelihood ratios, and `red_flags` that route to stop cards.

3.2 Signal model

- Each signal holds a **log-odds score** starting from a prior that can depend on demographics only once known (e.g. iron prior rises for menstruating users).
- An answer multiplies odds by the effect's likelihood ratio (`1r`). "Not sure" applies no update but marks the signal as *asked*.
- Lab values override questionnaire evidence: a ferritin result sets `low_iron_risk` directly from reference ranges and unlocks or removes iron.

- Signals are plain data ({name, logit, sources[]}); sources keeps the answer IDs so the UI can show *why* a signal exists.

3.3 Choosing the next question

1. **Hard gates first.** If a red flag fired → stop card. If the safety block (medications, conditions, pregnancy) hasn't run and the open-signal budget is spent → safety block.
2. **Candidate set.** All unanswered nodes whose preconditions pass.
3. **Score each candidate:** $\text{value} = \sum_s \text{goal_weight}(s) \times \text{expected_info_gain}(s \mid \text{node}) - \text{burden}(\text{node})$, where info gain uses the current probability and the node's effect table, goal_weight comes from the user's ranked goals, and burden penalises long or sensitive questions asked too early.
4. **Pick the max;** ties broken by phase order then editorial priority.
5. **Stop when** every signal relevant to a candidate ingredient is resolved ($p < 0.2$ or $p > 0.8$), or 30 cards are reached, or the user taps "Show my results" (allowed after the safety block).

This is what makes Maya's flow jump from "restless legs" to an iron cluster card to the diet question: each step had the highest expected gain for her top-ranked goals.

3.4 LLM-assisted follow-ups (bounded)

- **Free-text answers** ("anything else?") go to an LLM that maps text to existing signal IDs and option IDs with confidence; low-confidence mappings are shown back to the user as a confirm chip.
- **Rephrasing** for reading level or language uses the node's approved prompt as the source; output is checked for meaning drift by a second pass and falls back to the original text.
- **Generated follow-up questions** are allowed only from a whitelist of templates attached to a node (e.g. "How long has {symptom} lasted?"). The model can choose a template and fill slots; it cannot invent new medical questions.

3.5 Session state

IntakeSession {id, user_id?, graph_version, answers[], signals{}, asked[], phase, flags[], created_at} stored server-side; the client keeps a copy for offline resilience. Sessions are anonymous until the user saves results, then linked to an account. Every recommendation records the graph_version and rules_version it was built from.

4. Recommendation engine, knowledge base and safety rules

The engine turns signals into a stack in six deterministic stages. Each stage writes an audit record, so the UI's "why" text and the strike-through list on the analysis screen are generated from real decisions, not prose.

1. **Candidates** — every ingredient linked to an active signal or ranked goal through the evidence table.
2. **Eligibility** — remove anything excluded by diet, allergy, condition, medication, pregnancy/lactation or age. Each removal stores {ingredient, rule_id, reason_text} (Maya: collagen ← diet.vegetarian; St John's wort ← drug.hormonal_contraceptive).

3. **Lab locks** — ingredients flagged `requires_lab` (iron, high-dose vitamin D above the standard range, iodine) become *locked* unless a recent lab value is present.
4. **Dose** — pick a target from the ingredient's dose bands for the user's sex, age, weight band and signal strength; sum across all products for the same nutrient and cap at the tolerable upper intake level.
5. **Interactions & timing** — check ingredient↔ingredient and ingredient↔drug pairs; `major` removes the lower-value item, `moderate` adds a warning and spacing rule, `timing` only constrains the scheduler.
6. **Optimise** — choose the final set under the user's budget, pill limit and a max-items cap.

4.1 Optimisation

For each eligible ingredient i : $\text{value}_i = \sum \text{area_need_area} \times \text{benefit}_i, \text{area} \times \text{evidence_weight}_i \times \text{signal_fit}_i$. Selection is a small knapsack: maximise $\sum \text{value}$ subject to $\sum \text{cost} \leq \text{budget}$, $\sum \text{pills} \leq \text{limit}$, $\text{items} \leq 8$. With fewer than ~25 candidates an exact solver (OR-Tools CP-SAT) runs in milliseconds. Diminishing returns are applied per body area (each extra contributor counts less) so the stack doesn't pile five items onto sleep.

4.2 Knowledge base (Postgres, editor-managed)

Table	Key columns	Notes
<code>ingredient</code>	<code>id</code> , <code>name</code> , <code>forms[]</code> , <code>animal_derived</code> , <code>requires_lab</code>	Form matters: magnesium oxide ≠ bisglycinate
<code>evidence_claim</code>	<code>ingredient_id</code> , <code>body_area</code> , <code>outcome</code> , <code>population</code> , <code>effect_grade</code> (A–D), <code>effect_size</code> , <code>citations[]</code>	One row per claim; feeds benefit scores and "what research says" text
<code>dose_band</code>	<code>ingredient_id</code> , <code>form</code> , <code>population</code> , <code>min</code> , <code>typical</code> , <code>max</code> , <code>unit</code> , <code>with_food</code>	Typical = default target
<code>upper_limit</code>	<code>nutrient</code> , <code>population</code> , <code>ul_value</code> , <code>unit</code> , <code>source</code>	Summed across products
<code>contraindication</code>	<code>ingredient_id</code> , <code>condition_code</code> / <code>drug_class</code> / <code>diet</code> , <code>severity</code> , <code>rule_text</code> , <code>source</code>	Drives eligibility
<code>interaction</code>	<code>a_id</code> , <code>b_id</code> (ingredient or drug class), <code>severity</code> , <code>mechanism</code> , <code>spacing_hours</code> , <code>source</code>	Drives warnings + scheduler
<code>timing_rule</code>	<code>ingredient_id</code> , <code>prefer_slot</code> , <code>avoid_with[]</code> , <code>min_gap_hours</code> ,	Ashwagandha 8 on / 2 off

Table	Key columns	Notes
	cycle_on_weeks, cycle_off_weeks	
signal_ingredient	signal, ingredient_id, weight	Maps signals to candidates

Content sources. Public: NIH Office of Dietary Supplements fact sheets, Health Canada natural health product monographs, published RCTs and meta-analyses. Licensed (budget for these): a clinical interaction database (e.g. Natural Medicines or a DrugBank licence) for drug–supplement checks. Every row carries source, reviewed_by, reviewed_at; a pharmacist or registered dietitian must approve changes before a new rules_version ships.

4.3 Safety principles enforced in code

- Rules engine output is the only source of doses and exclusions; the LLM cannot add, remove or re-dose items.
- Red-flag answers always stop or restrict the flow; there is no override.
- Unknown medication (free text not matched to the drug database) → the plan shows a pharmacist-check banner and removes items with any drug-interaction rows.
- Restricted populations (pregnant, breastfeeding, under 18, kidney/liver disease) use a separate, smaller ingredient allow-list.
- Every plan is reproducible: (answers, labs, graph_version, rules_version, catalog_snapshot) → identical output, enforced by a regression suite.
- Payment status and affiliate data are never inputs to the rules engine; exclusions, interactions, locked items and stop cards are always shown free (section 13.4).
- Personal response rules (section 14.3) live in the same engine and go through the same clinical review as every other rule.

5. Health Impact scoring model

The Health Impact Map plots two numbers per body area on a 0–10 scale: the user's **need** and the stack's **projected benefit**, plus each supplement's share. All three come from stored data, so the chart is reproducible and explainable.

Need for area a comes from goal rank and the signals mapped to that area:

$$N_a = 10 \cdot \max\left(w_{\text{goal}}(a), \max_{\{s \in a\}} p_s\right)$$

Contribution of ingredient i to area a scales its normalised effect size by evidence grade and by how well the chosen dose matches the studied dose:

$$c_{\{i,a\}} = 10 \cdot b_{\{i,a\}} \cdot g_i \cdot f_i \cdot r_{\{i,a\}}$$

where b is the normalised effect size from evidence_claim (0–1), g the grade weight (A 1.0, B 0.75, C 0.5, D 0.25), f the dose fit (1 inside the studied range, falling linearly to 0 at half of it), and r the relevance to this user (1 if a matching signal is active, 0.5 for general benefit).

Combined benefit saturates, so stacking more items on one area gives diminishing returns:

$$S_a = 10 \cdot \left(1 - \prod_i \left(1 - \frac{c_{\{i,a\}}}{10}\right)\right) + \sum_{\{i,j\}} \sigma_{\{ij,a\}}$$

σ is a small synergy or antagonism term from the interaction table (bounded to ± 0.5). Coverage shown in the UI is $\min(S/N, 1)$. Each supplement's share of the bar is its c value scaled so the shares sum to S .

API output (feeds radar and bar views)

```
{ "plan_id": "pl_8f3", "model_version": "impact-1.2",
  "areas": [ { "area": "sleep", "need": 8.0, "projected": 5.5, "coverage": 0.69,
    "contributors": [ { "ingredient": "ashwagandha_ksm66", "share": 2.5, "grade":
      "B" },
      { "ingredient": "magnesium_bisglycinate", "share": 2.0,
        "grade": "B" } ],
    "gap_reason": null },
    { "area": "skin_hair", "need": 5.0, "projected": 2.0, "coverage": 0.4,
      "gap_reason": { "code": "locked_ingredient", "ingredient": "iron" } } ],
  "onset_weeks": { "sleep": [1, 3], "mood_stress": [4, 8] } }
```

Calibration loop

Check-ins (weeks 2, 4, 8) record the user's own 0–10 rating per area. The app shows *reported* change next to *projected*. Aggregated, de-identified pairs are used offline to recalibrate b and g per ingredient; the model version bumps and old plans keep their original version for reproducibility. Copy everywhere says "projected", never "will".

6. Dosage scheduling and calendar engine

The scheduler converts a stack into dated dose events. It is a small constraint solver over the user's daily slots, run every time the plan, routine or products change.

6.1 Inputs

- **Routine:** wake, breakfast, lunch, dinner, bed times, training days, IANA time zone (e.g. America/Toronto).
- **Per item:** dose and units per serving (from the chosen product), `prefer_slot`, `with_food`, `avoid_with[]` + `min_gap_hours`, frequency (daily, specific weekdays, alternate days), cycle (on/off weeks), split rule (e.g. split above a per-dose cap).
- **Plan settings:** start date, ramp-up interval (default: add 1–2 items every 3 days), events (race day: no new items).

6.2 Algorithm

1. Expand each item into daily dose units (respecting splits and frequency).
2. Assign units to slots with a CSP: hard constraints are food requirements, spacing (`avoid_with`), and bedtime/morning preferences; the objective minimises the number of distinct dose moments per day, then pills per moment.
3. Apply the ramp-up order, sorted by lowest side-effect risk first, so a reaction can be traced to one new item.
4. Lay cycles and events over the dates; generate check-in tasks (weeks 2, 4, 8) and lab tasks for locked items.

5. Compute refill dates: `runout = start + floor(units_in_bottle / daily_units);`
reminder 5 days earlier, carrying the product's affiliate link.

6.3 Output and sync

- Each recurring dose becomes an RRULE (RFC 5545) with a stable UID; changes bump `SEQUENCE`, so subscribed Google/Apple calendars update instead of duplicating.
- Times are stored as local wall-clock time + time zone, so a DST change keeps 21:30 at 21:30.
- Reminders: Web Push (VAPID) for the PWA, optional email digest; SMS only behind a paid add-on (cost).
- Adherence: `DoseLog {event_id, taken_at, status};` streaks and per-item adherence feed the check-in screen and the calibration loop.

6.4 Correctness tests

Property-based tests (Hypothesis / fast-check) generate random routines and stacks and assert: no spacing violation, no slot outside wake–bed, totals per day equal prescribed dose, cycles honoured, DST transitions keep local times.

7. Product catalog, market research pipeline and affiliate links

Products are matched to ingredients *after* the stack is decided, so commissions can never change what is recommended. The catalog service keeps a scored list of products per ingredient and form, refreshed from APIs and feeds, never by scraping retailer sites.

7.1 Data sources

Source	What it gives	Access
Amazon Creators API	ASIN, title, images, price, availability, rating, review count, affiliate URL	OAuth 2.0 client credentials. It replaced the Product Advertising API 5.0, which was deprecated on 30 April 2026 and retired on 15 May 2026 . Access reportedly needs 10 qualifying referred sales in the past 30 days, so launch with SiteStripe/search links and switch once eligible
iHerb affiliate feed	Product feed, prices, deep links	Via its affiliate network; commission reported around 5% base with tiers (details vary by source)
Brand programs	Direct links for brands sold mainly on their own sites (e.g. Transparent Labs, Thorne)	Individual applications; often higher rates than marketplaces

Source	What it gives	Access
Certification registries	NSF Certified for Sport, USP Verified, Informed Sport/Choice listings	Periodic import; match on brand + product + lot where listed
Label data	Supplement-facts panel: amounts, forms, excipients, capsule type	Manufacturer data + vision-model extraction from label images, human-verified
Expert roundups	Editorial "best of" picks used as a prior	Curated by the content team; LLM-assisted summarising with human review
Health Canada licence	Natural Product Number (NPN) for Canadian users	Licensed Natural Health Products Database lookup

7.2 Product score

$\text{score} = 0.30 \cdot \text{certification} + 0.25 \cdot \text{dose_fit} + 0.15 \cdot \text{form_fit} + 0.15 \cdot \text{rating_bayes} + 0.10 \cdot \text{price_per_effective_dose} + 0.05 \cdot \text{editorial}$

- `rating_bayes` shrinks small review counts toward the category mean so a 4.9★ with 40 reviews doesn't beat a 4.6★ with 30,000.
- `dose_fit` uses the user's target: a 5,000 IU softgel scores low for a 2,000 IU plan (this is why Maya's D3 pick differs from the most popular one).
- User filters (vegetarian capsule, no gelatin/soy/carrageenan, powders OK) are hard filters before scoring.
- Out of stock → next best is promoted and flagged in the UI.

7.3 Affiliate link router

1. Resolve the user's storefront from locale (amazon.ca for Canada, amazon.com for the US, iHerb as global fallback).
2. Choose the retailer per product: availability first, then price, then commission rate (only as a tie-breaker).
3. Build the URL with the right tag (`tag=<store-id>` for Amazon; network-specific parameters for others) and log `{user_hash, plan_id, product_id, retailer, ts}` on click.
4. Pull conversions from each network's reporting API nightly for revenue attribution.

Economics to plan for. Amazon's Health & Personal Care commission is about [1% in the US and about 3% on Amazon.ca](#), so brand and iHerb programs will usually earn more per sale; Amazon wins on trust and conversion. Model revenue per active user per month, not per click.

Compliance. A clear affiliate disclosure next to every buy button (FTC in the US, Competition Bureau in Canada); cache prices for at most 24 hours and show "price as of"; follow each program's rules on link redirects and on displaying ratings; never use affiliate data in the recommendation stage.

8. Frontend architecture

A single TypeScript web app, installable as a PWA so reminders work on phones without native apps in v1.

Concern	Choice	Why
Framework	Next.js (App Router) + React + TypeScript	SSR for marketing pages, client components for the intake, one deploy target
UI kit	Tailwind CSS + Radix primitives (shadcn/ui pattern)	Accessible components, design tokens from the prototype spec
State	TanStack Query (server state) + Zustand (intake session)	Intake survives refresh and flaky networks
Motion	Framer Motion	Card slide, insight-rail pulse, chart build-up
Charts	visx (D3 primitives) for radar + stacked bars	Full control of the custom radar and per-segment hover
Body map	Inline SVG with region IDs	Tappable, themeable, screen-reader labels
Calendar UI	Custom month/day views; ics generation server-side	Simple needs; avoids a heavy calendar lib
i18n	next-intl; English + French (Canada) at launch	Canadian market requirement
Analytics	PostHog, consent-gated; no health answers in events	Funnel + branch analytics without leaking health data
Testing	Vitest, Testing Library, Playwright, axe-core	Unit, component, end-to-end, accessibility

Route map

```
/                landing
/start           welcome + privacy promise
/intake/[sessionId]  one card at a time (client)
/intake/[sessionId]/review  "Here's what we heard"
/plan/[planId]    results: impact map, stack, products (gated sections redacted
server-side)
/plan/[planId]/calendar  today / month / export
/pricing          Free vs Full Report vs Plus
/checkout/success  post-purchase landing; claims the plan to the account
/account          saved plans, labs, settings
/account/billing  subscription, receipts, Stripe portal link
/account/privacy  consents, privacy history, export, delete
```

/go/[clickId] affiliate click logging (where program rules allow)

admin.stacksense.app (separate app, section 15)
/users /users/[id] /privacy-requests
/engine/graph /engine/knowledge /engine/releases /engine/simulator
/catalog/products /catalog/retailers /catalog/link-health
/revenue/plans /revenue/paywall /revenue/experiments /revenue/dashboards
/audit

Key components

QuestionCard (renders any node by `answer.type`), AnswerControl/* (chips, slider, time, body map, energy-curve drawer), InsightRail, ConfidenceRing, BranchToast, ImpactRadar, ImpactBars, StackCard, ProductRow, ExcludedDrawer, SafetyBanner, StopCard, DayTimeline, MonthGrid.

The client never decides what to ask: it posts an answer and renders whatever node the API returns. That keeps the question graph editable without frontend releases.

Revenue and privacy components: PaywallSheet (offers, preview, disclosure), LockedSection (renders a redacted payload with its unlock trigger), PricingTable, ConsentPanel (one toggle per purpose with plain-language text), PrivacyHistory, and ReturningUserBanner ("Welcome back — 3 quick check-ins before your updated plan"). Gate state always comes from the API's entitlements, never from client flags.

Accessibility: WCAG 2.2 AA; every chart has a data table alternative and an aria summary ("Sleep 69% covered"); colour is never the only signal (area icons + labels).

9. Backend services, data model and API contracts

Start as a **modular monolith** in Python (FastAPI) with clear module boundaries, one Postgres database and a job worker. Split modules into services only when load or team size demands it.

Module	Responsibility	Key tech
intake	Sessions, question graph traversal, signal updates	FastAPI, Pydantic, safe expression parser
rules	Eligibility, locks, dosing, interactions, optimisation, personal response rules	Pure Python + OR-Tools CP-SAT
impact	Health Impact scores	NumPy; versioned model
scheduler	Dose events, ramp-up, cycles, refills, .ics	python-dateutil RRULE, icalendar
catalog	Products, label data, certification matching, scoring, overrides	Postgres + nightly ingest jobs

Module	Responsibility	Key tech
affiliate	Retailer programs, link building, click log, conversion import, link health	Network APIs
billing	Plans, prices, checkout, webhooks, entitlements, paywall config, experiments	Stripe (Checkout, Billing, Tax, Customer Portal)
profile	Consent ledger, health-profile events and projections, user_features, privacy requests	Postgres event tables, KMS envelope encryption
llm	Phrasing, free-text mapping, explanations, evals	Model gateway with guardrails (section 10)
notify	Web Push, email, refill and check-in reminders	VAPID, transactional email provider
admin	/admin/v1 endpoints for the admin panel; roles, approvals, audit log	Role-based scopes, append-only audit table

Infrastructure: Postgres 16 (managed), Redis (sessions, rate limits, caches), job queue (Arq or Celery; Temporal if workflows grow), S3-compatible storage for label images and PDFs, all in a Canadian region (e.g. AWS ca-central-1) to simplify Canadian privacy obligations. Auth via a managed provider with passkeys and magic links. Infrastructure as code (Terraform); CI/CD on GitHub Actions.

9.1 Core data model

Table	Main columns
users	id, email (encrypted), locale, tz, stripe_customer_id, created_at
intake_sessions	id, user_id?, graph_version, phase, signals jsonb, flags[], status
answers	session_id, node_id, node_version, value jsonb, answered_at
labs	user_id, analyte (ferritin, b12, 25-OH-D), value, unit, drawn_at, source
plans	id, user_id, session_id, rules_version, impact_version, catalog_snapshot_id, status, created_at

Table	Main columns
plan_items	plan_id, ingredient_id, form, dose, unit, state (active / locked / excluded), reason_codes[]
plan_item_products	plan_item_id, product_id, rank, retailer, affiliate_url
dose_events	plan_id, item_id, rrule, local_time, tz, start_date, end_date, uid, sequence
dose_logs	event_id, occurred_on, status, logged_at
checkins	plan_id, week, area_scores jsonb, side_effects jsonb
products	id, brand, name, gtin, asin?, npn?, form, serving_units, certs[], rating, review_count, price, currency, updated_at
product_ingredients	product_id, ingredient_id, amount, unit, form
product_overrides	product_id, action (pin / demote / ban), reason, actor, expires_at
retailer_programs	id, storefront, network, tag_ref (vault key), commission_rate, cookie_days, link_template, active
clicks / conversions	click_id, plan_id, product_id, retailer, ts / network_order_id, amount, commission
consents	user_id, purpose, granted, policy_version, text_hash, method, ts (append-only)
profile_events	user_id, type (answer / lab / dose / checkin / side_effect / preference), payload (encrypted), ts
user_features	user_id, adherence jsonb, outcome_deltas jsonb, side_effect_flags[], computed_at
privacy_requests	id, user_id, type (export / delete / correct), status, due_at, completed_at
billing_plans / prices	plan key, feature_keys[] / stripe_price_id, currency, region, amount, interval
purchases / subscriptions	user_id, stripe ids, plan key, status, current_period_end, cancel_at

Table	Main columns
entitlements	user_id, feature_key, source (purchase / subscription / grant), expires_at
paywall_configs	version, gates jsonb, triggers jsonb, copy jsonb, status
experiments / assignments	key, variants, metrics, status / user_id, variant, assigned_at
admin_users / audit_log	id, role, sso_subject / actor, action, target, before, after, reason, ts
rules_releases	rules_version, status, author, reviewer, rollout_pct, published_at
Knowledge tables	see section 4.2

9.2 API (REST, JSON, versioned under /v1)

Method + path	Purpose
POST /intake/sessions	Start; returns {session_id, node} (prefilled from the profile for returning users with personalisation consent)
POST /intake/sessions/{id}/answers	Submit {node_id, value}; returns {next: node review stop, toast?, signals_delta[], confidence}
GET /intake/sessions/{id}/review	Signals with their source answers
PATCH /intake/sessions/{id}/signals	User removes or confirms a signal
POST /plans	Build a plan from a session; returns stack, exclusions, locks
GET /plans/{id}	Plan with gated fields redacted per the caller's entitlements
GET /plans/{id}/impact	Health Impact Map payload (section 5), redacted for free users
GET /plans/{id}/products	Ranked products per item for the user's storefront
GET /plans/{id}/schedule?from&to	Dose events for a date range (7 days on Free)

Method + path	Purpose
GET /plans/{id}/calendar.ics?token=	Subscribable feed (signed token; requires calendar_90d or calendar_ongoing)
POST /plans/{id}/labs	Add lab values; triggers re-plan and unlocks
POST /dose-logs, POST /checkins	Adherence, outcome and side-effect tracking
POST /clicks	Log affiliate click, return destination URL
GET /billing/offers	Plans, prices and paywall config for the user's region and experiment variant
POST /billing/checkout	Create a Stripe Checkout session for a plan key
POST /billing/portal	Return a Stripe Customer Portal link
POST /webhooks/stripe	Verified, idempotent webhook handler → entitlements
GET /me/entitlements	Feature keys for the current user
GET /me/consents, PUT /me/consents/{purpose}	Read and change consent per purpose
GET /me/profile, POST /me/privacy/export, POST /me/privacy/delete	Profile view and self-serve privacy requests
/admin/v1/*	Admin panel endpoints (users, engine, catalog, revenue, audit), role-scoped; see section 15

Example answer exchange from Maya's B3 card:

```
// request
{ "node_id": "B3_diet", "value": { "choice": "vegetarian", "years": 6 } }
// response
{ "next": { "id": "B4_menstruation", "prompt": "Do you have menstrual periods?",
  "why": "Periods are the most common cause of low iron in your age group."
},
  "toast": "Because you're vegetarian, we'll check B12 and iron next.",
  "signals_delta": [ { "signal": "b12_risk", "p": 0.71 }, { "signal": "omega3_gap",
    "p": 0.8 } ],
  "exclusions_added": [ { "ingredient": "collagen_peptides", "reason":
    "animal_derived" } ],
  "confidence": 0.48 }
```

OpenAPI is the contract; the frontend generates its typed client from it.

10. AI / LLM layer: prompts, guardrails and evals

The LLM makes the product feel conversational and personal; it never makes medical decisions. Five jobs, each with a strict input/output contract.

Job	Input	Output (JSON schema)	Fallback
Free-text → signals	User text + list of allowed signal/option IDs	[{signal_id, option_id, confidence, quote}]	Ask the user to pick from chips
Question rephrasing	Approved node text + reading level / locale	{prompt, helper}	Approved text
Template follow-ups	Node's template list + answer so far	{template_id, slots} or null	Skip follow-up
"Why you" + explanations	Plan JSON, user's answers, evidence claims for each item	{item_id, why_you, what_it_does, evidence_summary}	Templated sentence from reason codes
Doctor note summary	Plan + signals + locks	Structured sections for the PDF	Plain templated PDF

Prompt pattern (explanations)

SYSTEM: You write short, plain-language explanations for a supplement plan. Use ONLY the facts in <plan>, <answers> and <evidence>. Do not add supplements, change doses, or claim to diagnose, treat or cure. Quote the user's own answers when saying why an item was chosen. Grade 7 reading level. Return JSON matching the schema. If a fact you need is missing, return "insufficient_context".
 <plan>{...}</plan> <answers>{...}</answers> <evidence>{...}</evidence>

Guardrails

- **Structured output only**, validated with the JSON schema; invalid → one retry → fallback.
- **Faithfulness checker**: every number, ingredient name and dose in the output must exist in the plan JSON; any mismatch rejects the output.
- **Claims filter**: blocks disease-treatment language ("cures", "treats anxiety") that supplement rules don't allow, and adds the standard disclaimer.
- **Data minimisation**: send signal IDs and short answer labels, not names, emails or raw lab reports; use a provider configuration with no training on customer data and the shortest available retention.
- **Model gateway**: one internal interface with per-job model choice (a smaller fast model for mapping and rephrasing, a stronger one for explanations), timeouts, cost caps and logging of prompt/response hashes.

Evals (run in CI before any prompt or model change)

- **Golden personas**: 300+ scripted users (including Maya, pregnant users, users on SSRIs or blood thinners, vegans, athletes, people with red-flag answers) with expected stacks, exclusions and stop cards. The rules engine must match 100%.
- **LLM metrics**: free-text mapping F1 ≥ 0.9; explanation faithfulness 100% on numbers; reading grade ≤ 8; forbidden-claim rate 0.

- **Simulated users:** an LLM role-plays personas through the live intake to find dead ends, loops and confusing questions; failures become new graph tests.
- **Clinician review:** a monthly sample of 50 real (de-identified) plans reviewed by a pharmacist; findings feed rule changes.

11. Compliance, privacy and security

StackSense handles consumer health data and earns affiliate revenue, so privacy and claims compliance are launch blockers, not polish. Budget for a privacy lawyer and a regulatory review before public launch; the list below is the engineering baseline, not legal advice.

Regulatory map (verify with counsel)

Area	Canada	United States
Privacy	PIPEDA; Quebec Law 25 (privacy impact assessment, explicit consent, notice if data leaves Quebec)	FTC Act + Health Breach Notification Rule for non-HIPAA health apps; state consumer-health-data laws (e.g. Washington's My Health My Data Act) and CPRA sensitive data rules
Health claims	Natural health product rules; only claims the product licence allows	DSHEA structure/function claims only, no disease claims
Medical device line	Stay a wellness / information tool; no diagnosis	Same; avoid features that read as diagnosing a condition
Advertising	Competition Bureau: clear affiliate disclosure	FTC Endorsement Guides: clear and conspicuous disclosure

Privacy by design

- **No ad-tech on health pages.** No third-party advertising pixels anywhere in the intake or plan; analytics events carry no answer content.
- **Consent:** explicit, granular consent before storing answers; separate opt-in for using de-identified data to improve the model.
- **Anonymous first:** intake works without an account; data is linked only when the user saves.
- **User rights:** export (JSON + PDF) and delete-everything in two clicks; deletion propagates to backups within the retention window.
- **Retention:** abandoned anonymous sessions deleted after 30 days; click logs keep a hashed user ID only.

Security baseline

- TLS everywhere; encryption at rest plus field-level encryption (KMS) for answers, labs and medications.
- Role-based admin access with SSO and MFA; every editor change and every data access by staff written to an append-only audit log.
- OWASP ASVS level 2 as the checklist; dependency and container scanning in CI; secrets in a managed vault.
- Rate limits and bot protection on intake and click endpoints (affiliate fraud protection).
- Annual third-party penetration test; incident response runbook with breach-notification timelines for each jurisdiction.

Payments and subscriptions

- Card data never touches our servers: Stripe Checkout and the Customer Portal handle payment details, which keeps PCI scope to the simplest self-assessment level.
- Subscription rules: show price, billing interval, renewal and trial end date before purchase; send a reminder before a trial converts and before annual renewals; cancellation as easy as sign-up. Check Canadian provincial consumer-protection rules and US state auto-renewal laws with counsel.
- Taxes: GST/HST/QST and US sales tax calculated by Stripe Tax; receipts show tax and business number.
- Refunds and chargebacks handled in the admin panel and logged; repeated refunds flag accounts for review.

Consent and admin access

- Health data is stored only with `profile_storage` consent, used for personalisation only with `personalisation` consent, and used for research only de-identified and with `research` consent (section 14).
- Staff access to identifiable health data is reason-logged and visible to the user in their privacy history (section 15.2).

12. Testing, observability, delivery plan and team

12.1 Testing pyramid

- **Rules golden tests:** every persona in the eval set produces an exact expected plan; any diff fails CI and needs clinician sign-off.
- **Property tests:** scheduler (section 6.4), upper-limit summation, optimiser never exceeding budget or pill limit.
- **Graph tests:** no unreachable nodes, no loops, every red flag reachable, every ingredient reachable by at least one path.
- **Contract tests:** OpenAPI schema checked on both sides; affiliate URL builder tested per storefront.

- **Entitlement tests:** every gated endpoint returns redacted data without the feature key; safety fields are present for every tier; Stripe webhook replays and out-of-order events leave entitlements correct.
- **Consent tests:** no profile data read or written without the matching purpose; withdrawal removes data within 24 hours; research exports contain no direct identifiers.
- **Admin tests:** each role can reach only its scopes; unapproved rules cannot be published; every write lands in the audit log.
- **End-to-end:** Playwright runs Maya's path (free, report buyer, Plus subscriber, returning user) plus 10 other personas on every pull request; axe-core accessibility checks.

12.2 Observability

OpenTelemetry traces across modules; Sentry for errors; product metrics in PostHog (consent-gated). Dashboards: intake completion by card, branch frequency, drop-off after sensitive questions, plan build latency, LLM fallback rate, faithfulness rejections, click-through and conversion by retailer, reminder delivery rate. Alerts on any safety-rule regression or a spike in LLM rejections.

12.3 Delivery plan (about 19 weeks to closed beta)

1. **Weeks 1–2 — Foundations:** repo, CI/CD, auth, Postgres schema (including consent and audit tables), design tokens, clickable prototype from the UI spec tested with 8–10 users.
2. **Weeks 3–6 — Intake engine:** question graph format, signal model, next-question selection; first 120 nodes authored with a pharmacist or dietitian.
3. **Weeks 5–9 — Rules + impact:** knowledge base for the first 40 ingredients, eligibility, locks, dosing, interactions, optimiser, impact scores; golden persona suite.
4. **Weeks 8–11 — Results + calendar:** impact map, stack cards, scheduler, .ics, Web Push, check-ins.
5. **Weeks 10–13 — Catalog + affiliate:** Amazon search links → Creators API when eligible, iHerb + 2–3 brand programs, product scoring, click tracking.
6. **Weeks 11–14 — Profile + consent:** consent ledger, health-profile events, returning-user intake, personal response rules, privacy self-service.
7. **Weeks 12–15 — Billing + payroll:** Stripe products, checkout, webhooks, entitlements, redacted payloads, pricing page, payroll sheet.
8. **Weeks 12–16 — Admin panel:** roles and audit first, then engine release workflow and simulator, catalog and link health, revenue and payroll settings, user and privacy tools.
9. **Weeks 14–16 — LLM layer:** mapping, explanations, guardrails, eval harness.
10. **Weeks 17–19 — Hardening:** privacy review, pen test, clinician review of 100 plans, billing test cycles, load test, closed beta.

12.4 Team and roles

Role	Owns
Product lead	Scope, metrics, pricing strategy, affiliate partnerships
Clinical lead (pharmacist or RD, part-time)	Question graph content, evidence and rules sign-off; clinical reviewer role in admin

Role	Owns
Tech lead / backend	Engine, rules, scheduler, billing, consent, data model, security
Frontend engineer	PWA, intake UI, charts, calendar, paywall, admin panel UI
Designer	Prototype, design system, paywall and pricing tests, admin usability
Content/research analyst	Product catalog curation, certification matching, link health
Privacy and legal counsel (contract)	Consent text, privacy policy, subscription terms, regulatory review

12.5 Building it with AI coding agents

- Keep this document and the prototype spec in the repo as `/docs`; add an `AGENTS.md` (or `CLAUDE.md`) that states the hard rules: the rules engine is the only source of doses; never edit `knowledge/` data without a linked clinician approval; every change ships with tests.
- Break work into tickets that each name the module, the API contract affected and the acceptance tests (e.g. "Implement `POST /intake/sessions/{id}/answers` so Maya's B3 answer returns the response in 9.2").
- Let agents write code, tests and migrations; keep humans as reviewers for anything touching `rules/`, `knowledge/`, privacy or affiliate compliance.
- Use the golden personas as the agents' definition of done: a pull request is mergeable only when the suite passes and the diff of any plan output is explained.

13. Revenue model and paywall

StackSense earns from three streams: a one-time **Full Report**, a **Plus** subscription for ongoing tracking, and **affiliate commissions** on every tier. The intake, the stack names and all safety information stay free; the paywall sits on depth and ongoing value, never on safety. Every price, gate and placement below is a default the revenue manager can change in the admin panel (section 15) without a release.

13.1 Tiers (launch defaults, CAD, to be tested)

Feature	Free	Full Report (one-time, \$9.99)	Plus (\$5.99/month or \$49/year)
Adaptive intake and "Here's what we heard"	Yes	Yes	Yes
Stack names + one-line "why you"	Yes	Yes	Yes

Feature	Free	Full Report (one-time, \$9.99)	Plus (\$5.99/month or \$49/year)
Exclusions, interactions, locked items, stop cards	Always	Always	Always
Health Impact Map	Top 3 areas; rest blurred	Full map + contributors	Full map + reported vs projected over time
Exact doses and timing	Ranges only	Exact	Exact
Product picks with affiliate links	Best match per item	All alternatives + "why this product"	Same + price-drop alerts
Dosage calendar	Today view, 7 days	90-day calendar + .ics export	Ongoing calendar, push reminders, refill reminders
Doctor note PDF	—	Yes	Yes
Check-ins, lab upload and automatic re-plan	—	—	Yes
Re-run intake with saved profile (section 14)	—	One re-run within 90 days	Unlimited

Rationale: free users still see a useful, safe plan and generate affiliate revenue; the report converts people who want the full picture; Plus monetises the calendar, reminders and personalisation, which are what bring people back.

13.2 Paywall behaviour

- **Triggers** (configurable): tapping a blurred area on the Impact Map, "See exact doses", "Open full calendar", "Download doctor note", or a soft prompt after the results page has been open for 60 seconds.
- **Paywall sheet**: what's inside (a real preview of Maya-style content with her own numbers blurred, not stock copy), the two offers side by side, trust line ("Safety information is always free"), affiliate disclosure, and "Not now" that always works.
- **Server-side enforcement**: the API returns redacted payloads for gated fields ("doses": {"locked": true, "range": "200-400 mg"}). Gated data is never sent to the browser and hidden with CSS.
- **Trials and promos**: 7-day Plus trial (card required, reminder 2 days before charge), coupon codes, partner codes for clinicians or gyms.
- **Experiments**: price points, which sections are gated, paywall copy and trigger timing are A/B-testable per region from the admin panel; assignment is sticky per user.

13.3 Entitlements and billing

- **Payments:** Stripe Checkout for one-time and subscription purchases (CAD and USD), Customer Portal for self-serve cancel/update, Stripe Tax for GST/HST and US sales tax.
- **Entitlements service:** Stripe webhooks (`checkout.session.completed`, `invoice.paid`, `customer.subscription.updated/deleted`, `charge.refunded`) update an entitlements table; the API checks `has_feature(user, "impact_full")` on every gated read. Webhooks are verified, idempotent and replayable.
- **Feature keys:** `impact_full`, `exact_doses`, `calendar_90d`, `calendar_ongoing`, `reminders`, `doctor_note`, `product_alternatives`, `checkins`, `lab_replan`, `rerun_intake`. Plans map to feature sets in admin, so a new bundle needs no code.
- **Anonymous purchase:** buying the report creates an account (magic link) so the purchase isn't lost; the session's answers attach to that account only with the user's consent (section 14).
- **Cancellation:** one click in the account page or the portal, effective at period end; access to a purchased Full Report never expires.

13.4 Integrity rules

- Payment status never changes the recommendation, the order of products, or what safety information is shown.
- No paid placement inside recommendations. If brand partnerships are added later, they appear only in a separate, labelled "Sponsored" area, never in the stack or product ranking.
- Personal or health data is never sold or shared with advertisers or brands.

13.5 Unit economics (illustrative assumptions to replace with real data)

Input	Assumption
Completed intakes per month	10,000
Buy Full Report	4% at \$9.99 → about \$4,000
Start Plus (after trial)	2% at \$5.99/month → 200 new subscribers
Affiliate	25% click a product, 30% of those buy, \$60 average order, 4% blended commission → about \$1,800
Payment fees	about 3% of card revenue

With monthly churn around 8%, Plus subscribers accumulate toward roughly 2,500 at steady state, so subscription revenue overtakes the report within the first year. The admin revenue dashboard (section 15.5) tracks each input so these assumptions are replaced with measured values within weeks of launch.

14. User data, consent and personalisation

Every answer, plan, dose log and check-in is stored as a versioned, consent-tagged health profile, so the next recommendation starts from what we already know and from what actually worked for this person.

Nothing is kept beyond the anonymous session unless the user opts in, and each use of the data has its own consent.

14.1 Consent purposes

Purpose key	What it allows	Default	If withdrawn
profile_storage	Keep answers, labs, plans and logs in the user's account	Asked when saving results	Account health data deleted; billing records kept as required by tax law
personalisation	Use stored history to tailor future questions and plans	Asked with profile_storage, separate box	Future plans built from the current session only
reminders	Push and email reminders, refill and check-in prompts	Asked when the calendar is first opened	Reminders stop
research	Use de-identified data to improve scoring models	Off; explicit opt-in	Excluded from future training sets
marketing	Product news and offers by email	Off; explicit opt-in	Unsubscribed

Each grant or withdrawal is written to an append-only **consent ledger**: {user_id, purpose, granted, policy_version, text_hash, method (checkbox / settings / support), ts}. Services read consent through one `consent.allows(user, purpose)` check; a background job enforces withdrawals within 24 hours. Users see and change all purposes in Account → Privacy.

14.2 What the health profile stores

Layer	Contents	Why it matters later
Stable facts	Diet, allergies, conditions, medications, sex, birth year, routine	Not re-asked; confirmed with one "still true?" card
Answer history	Every session's answers with node versions	Shows change over time (stress 9 → 5)
Signal history	Signal probabilities per session	Detects trends (sleep signal resolving)
Labs	Analyte, value, unit, date, source	Unlocks items; sets doses; triggers re-test reminders

Layer	Contents	Why it matters later
Plans	Every plan with rules, impact and catalog versions	Reproducible history; "what you took when"
Adherence	Dose logs per item	Distinguishes "didn't work" from "wasn't taken"
Outcomes	Check-in scores per body area; side-effect reports per item	Feeds personal re-planning and model calibration
Preferences	Budget, pill limit, forms, brands liked or avoided, product ratings	Shapes product picks and stack size

14.3 How stored data personalises the next recommendation

- **Shorter, smarter re-intake:** the graph starts from stored signals; stable facts show as confirm chips; the engine asks only about changes and unresolved signals (Maya's second visit opens with "Did you get your ferritin tested?").
- **Personal response rules** (run in the rules engine, so they stay auditable):
 - o Item taken $\geq 80\%$ of days for 8+ weeks with no improvement in its target area → suggest a swap or removal.
 - o Side effect reported → exclude that ingredient or form for this user (e.g. switch magnesium from capsule to powder, or to a different form).
 - o Adherence below 50% for a slot → move the item to a slot she does keep, or reduce moments per day.
 - o Lab value entered → unlock, re-dose or retire items (iron unlocks at a low ferritin; vitamin D dose adjusts to her level).
 - o Season and location → vitamin D and immunity weights rise from October to March for northern users.
- **Product memory:** products she rated poorly or found hard to swallow are filtered out; liked brands get a small boost within the same certification tier.

14.4 Storage design

- **Separation:** identity (users, email, billing) and health data live in different schemas, joined only by an internal pseudonymous ID; staff tools see health data masked by default.
- **Encryption:** field-level encryption with per-user data keys wrapped by a KMS master key; deleting the user's data key makes any backup copies unreadable (crypto-shredding).
- **Event-sourced history:** answers, labs, dose logs and check-ins are append-only events; the current profile is a projection rebuilt from them. This gives a full audit trail and simple point-in-time views.
- **Personalisation features:** a nightly job computes `user_features` (adherence per item, outcome deltas, side-effect flags, season) used by the rules engine; only users with personalisation consent are included.

- **Research warehouse:** for research consenters only, a de-identified copy (no names, emails or free text; dates shifted, ages banded, rare values suppressed) goes to an analytics warehouse for model calibration.
- **Retention:** anonymous sessions 30 days; inactive accounts reminded at 24 months and deleted at 36 unless renewed; billing records kept for the period tax law requires.
- **User rights:** export everything (JSON + PDF) and delete everything from Account → Privacy; requests complete automatically, with an admin queue only for edge cases.

15. Admin panel

The admin panel is a separate web app (`admin.stacksense.app`) on the same API under `/admin/v1`, with four workspaces — Users & data, Recommendation engine, Affiliate & catalog, Revenue & payroll — plus shared audit and analytics. Every change is role-checked, versioned and logged; anything that can affect a recommendation goes through clinical review before it reaches users.

15.1 Roles and access

Role	Can do	Cannot do
Super admin	Manage staff, roles, global settings	Publish rules without a clinical approver
Clinical editor	Draft question nodes, evidence, rules, dose bands	Approve own changes
Clinical reviewer (pharmacist / RD)	Approve or reject drafts, publish rules versions	Change billing or affiliate settings
Catalog manager	Products, retailers, affiliate tags, product overrides	Edit rules or see health data
Revenue manager	Prices, plans, payroll gates, promos, experiments	See identifiable health data
Support agent	Look up accounts, grant/refund entitlements, run privacy requests	See unmasked health data without a logged reason
Analyst	Dashboards on de-identified data	Access individual records

Security: SSO with enforced MFA, short sessions, optional IP allowlist, least-privilege scopes on every `/admin/v1` endpoint, and an append-only audit log (`actor`, `role`, `action`, `target`, `before`, `after`, `reason`, `ts`) viewable and exportable by super admins.

15.2 Users & data

- **Search** by email, user ID, order ID or click ID. Results show account, plan status, subscription, consents and last activity.

- **User record:** profile layers from section 14.2 shown masked; "Reveal health data" requires choosing a reason (support ticket number) and is logged and visible to the user in their privacy history.
- **Consents:** full consent-ledger timeline; support can record a consent change requested by email, never grant one silently.
- **Plans:** every plan with the rules/impact/catalog versions used and a "re-run with current rules" preview (read-only diff) to answer "why did my plan change?".
- **Privacy requests:** queue for export, deletion and correction with due dates per jurisdiction; one-click fulfilment that calls the same jobs users trigger themselves.
- **Account actions:** resend magic link, merge duplicate accounts, grant or revoke entitlements, issue refunds (passes through to Stripe), suspend for abuse.
- **Cohort tools (de-identified):** counts by signal, goal, stack item and outcome for analysts; no export of identifiable rows.

15.3 Recommendation engine

- **Question graph editor:** visual node map with preconditions, effects and red flags; edit a node in a form with live preview of the card; graph checks (unreachable nodes, loops, missing translations) run on save.
- **Knowledge editors:** ingredients, evidence claims, dose bands, upper limits, contraindications, interactions, timing and cycling rules (section 4.2 tables), each with source citations required.
- **Signal ↔ ingredient weights and scoring parameters:** impact-model weights (section 5) and optimiser settings (max items, diminishing-returns factor) editable as versioned config.
- **Release workflow:** Draft → Automated checks (golden personas, graph tests) → Clinical review → Approved → Published to a % of new plans → 100%. One-click rollback to any previous `rules_version`.
- **Simulator:** pick a golden persona or build one with sliders, run it against *draft* and *live* side by side, and see the plan diff (items added/removed, dose changes, exclusions) before approving.
- **Impact report for a change:** how many existing users would get a different plan, by how much, and which safety rules are touched; existing plans are never silently changed — users get a "your plan has an update" notice instead.

15.4 Affiliate & catalog

- **Retailers & programs:** per storefront (amazon.ca, amazon.com, iHerb, brand sites) store tag or partner ID, commission rate, cookie window, link template, redirect rules and API credentials (in the secrets vault, never shown in full).
- **Products:** create/edit products, map to ingredients and forms, edit label data, attach certification evidence, view score breakdown (section 7.2).
- **Overrides:** pin, demote or ban a product for a reason (quality issue, recall, supplier problem). Overrides change product order only, never which ingredient is recommended, and require a written reason.
- **Link health:** nightly checker for broken links, out-of-stock items, price jumps and missing tags; alerts and an auto-fallback to the next product.
- **Performance:** clicks, conversions, revenue and EPC by retailer, product and ingredient; reconciliation against network reports.

15.5 Revenue & paywall

- **Plans & prices:** create products (Full Report, Plus monthly/annual, bundles) with prices per currency and region, synced to Stripe Prices.
- **Feature gates:** a matrix of feature keys × plans (section 13.3) with a locked row for safety features that cannot be gated.
- **Paywall placements:** choose triggers, copy, preview image and order of offers; preview on mobile and desktop before publishing.
- **Promos & trials:** coupon codes, partner codes with attribution, trial length per region.
- **Experiments:** A/B tests on price, gates, copy and timing with sticky assignment, guardrail metrics (refund rate, complaint rate) and automatic stop on harm.
- **Dashboards:** funnel (intake started → completed → paywall viewed → purchased), MRR, ARPU, churn, trial conversion, refunds, affiliate revenue, revenue per completed intake.

15.6 Build notes

Next.js + the same component library as the main app, data tables with server-side filtering, and forms generated from the JSON schemas used by the API, so new rule types appear in the editor without custom UI. All admin writes go through the same domain services as the app, never direct database edits.

Sources

- [Amazon PA-API v5 deprecation and Creators API migration](#)
- [Product Advertising API 5.0 documentation notice](#)
- [Amazon Associates commission rates by country](#)
- [Amazon affiliate commission rates by category, 2026](#)
- [iHerb affiliate program details](#)
- Product picks and their sources are listed in the prototype spec, section 7.